COS30018 - Intelligent Systems

Option C - Task 1: Setup

Student Name: Le Minh Kha

Student ID: 104179506

Table of Contents

1. Environment Setup
2. Testing v0.1 Code Base
3. Testing P1 Code Base
4. Performance Comparison
5. Understanding v0.1
6. Conclusion

1. Environment Setup

1.1 Virtual Environment Creation

The project was set up using a Python virtual environment to ensure dependency isolation and reproducibility. The following steps were executed:

1. python -m venv venv
2. venv\Scripts\activate (Windows)
3. pip install -r requirements.txt

1.2 Requirements File

The requirements.txt file contains all necessary dependencies for both v0.1 and P1 implementations:

Package	Version
numpy	1.24.3
matplotlib	3.7.1
pandas	2.0.2
tensorflow	2.13.0
scikit-learn	1.3.0
pandas-datareader	0.10.0
yfinance	0.2.28
protobuf	3.20.3

1.3 Installation Process

All dependencies were successfully installed without conflicts. The protobuf version was specifically set to 3.20.3 to ensure compatibility with TensorFlow 2.13.0.

2. Testing v0.1 Code Base

2.1 Overview

v0.1 is a simple LSTM-based stock prediction model that was created following a YouTube tutorial. It predicts stock prices using only the closing price feature and implements a basic 3-layer LSTM architecture.

2.2 Key Characteristics

- Stock: CBA.AX (Commonwealth Bank of Australia)
- Features: Close price only (1 feature)
- Training Period: 2020-01-01 to 2023-08-01
- Testing Period: 2023-08-02 to 2024-07-02
- Architecture: 3 LSTM layers with 50 units each
- Dropout: 0.2 after each LSTM layer
- Optimizer: Adam
- Loss Function: Mean Squared Error (MSE)
- Training Epochs: 25
- Batch Size: 32
- Prediction Window: 60 days

2.3 Training Process

The model was trained for 25 epochs, taking approximately 2-3 minutes to complete. Training progress showed consistent loss reduction. Figure 2 shows the complete training output including epoch progress, loss values, and final performance metrics.

```
PS D:\COS30018-Intelligent Systtem\w3> python stock_prediction_v01.py
27/27 ━━━━━━━━━━━━━━━━━━━━ 2s 58ms/step - loss: 0.0053
Epoch 25/25
27/27 ━━━━━━━━━━━━━━━━━━━━ 2s 55ms/step - loss: 0.0048
[*********************100%***********************]  1 of 1 completed
8/8 ━━━━━━━━━━━━━━━━━━━━ 1s 107ms/step

============================================================
PERFORMANCE METRICS - v0.1
------------------------------------------------------------
Mean Absolute Error (MAE):      $3.71
Root Mean Squared Error (RMSE): $4.56
Mean Squared Error (MSE):       20.81
Mean Absolute Percentage Error: 3.39%
R² Score:                       0.7470
============================================================

Graph saved as 'v01_results.png'
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 59ms/step

Next Day Prediction: $113.18
```

Figure 2: v0.1 Training Progress and Performance Metrics

2.4 Performance Metrics

The model achieved the following performance metrics on the test set:

Metric	Value
Mean Absolute Error (MAE)	$3.71
Root Mean Squared Error (RMSE)	$4.56
Mean Squared Error (MSE)	20.81
Mean Absolute Percentage Error (MAPE)	3.39%
R^2 Score	0.7470

Figure 3 shows the actual versus predicted stock prices for CBA.AX. The smooth green prediction line closely follows the general trend of actual prices (black line), demonstrating good generalization.

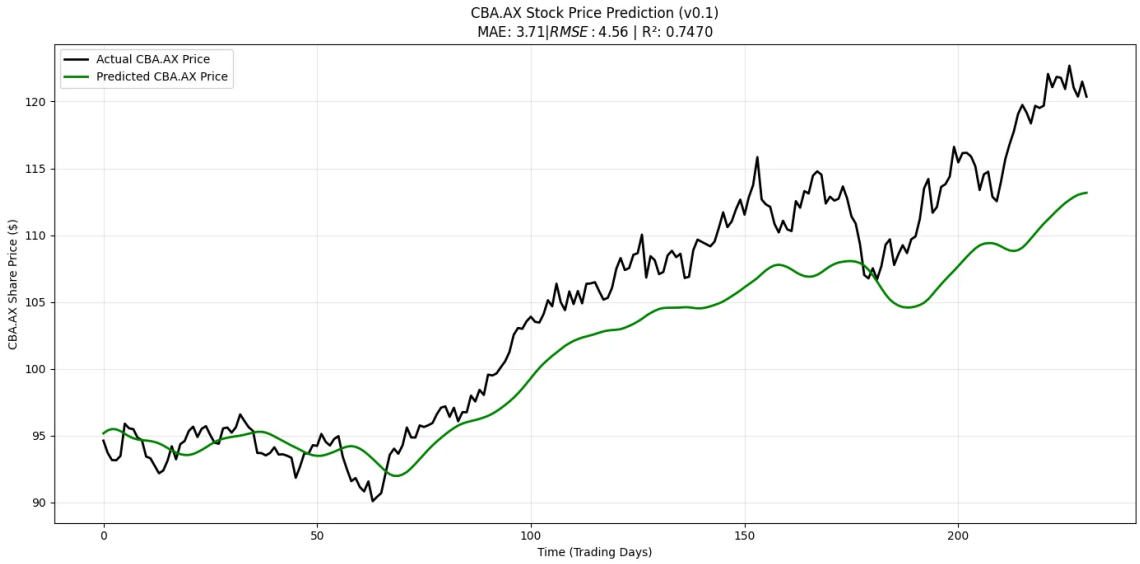

Figure 3: v0.1 Prediction Results - Smooth Trend-Following Behavior

2.5 Analysis

v0.1 demonstrates strong predictive performance with an MAPE of 3.39%, meaning predictions are on average within 3.39% of actual values. The R^2 score of 0.7470 indicates the model explains approximately 75% of the variance in stock prices.

Most importantly, the visualization reveals smooth, trend-following predictions that capture the overall price movement without fitting to short-term noise. This suggests

good generalization - the model has learned meaningful patterns rather than memorizing training data.

Next Day Prediction: $113.18

3. Testing P1 Code Base

3.1 Overview

P1 is a more sophisticated implementation from GitHub that uses multiple features and a more powerful LSTM architecture. It implements proper train/validation splitting and uses callbacks for model checkpointing.

3.2 Key Characteristics

- Stock: AMZN (Amazon)
- Features: adjclose, volume, open, high, low (5 features)
- Data Period: Maximum available historical data
- Training Samples: 5,718
- Testing Samples: 1,430
- Architecture: 2 LSTM layers with 256 units each
- Dropout: 0.4 after each LSTM layer
- Optimizer: Adam
- Loss Function: Huber Loss
- Training Epochs: 50
- Batch Size: 64
- Prediction Window: 50 days
- Lookup Step: 15 days ahead

3.3 Training Process

The model was trained for 50 epochs with validation monitoring, taking approximately 10-15 minutes. The training process showed validation loss and mean absolute error for each epoch, allowing for early stopping if needed. The best model weights were automatically saved using ModelCheckpoint callback. Figure 4 shows the final epoch results and model summary.

```
PS D:\COS30018-Intelligent Systtem\w3> python train_P1.py
90/90 ━━━━━━━━━━━━━━━━━━━━ 0s 140ms/step - loss: 4.4743e-04 - mean_absolute_error: 0.0181
Epoch 50: val_loss did not improve from 0.00026
90/90 ━━━━━━━━━━━━━━━━━━━━ 14s 155ms/step - loss: 4.4771e-04 - mean_absolute_error: 0.0181 - val_
e_error: 0.0128

============================================================
Making predictions...
============================================================
45/45 ━━━━━━━━━━━━━━━━━━━━ 3s 64ms/step

============================================================
PERFORMANCE METRICS - P1
============================================================
Mean Absolute Error (MAE):      $0.01
Root Mean Squared Error (RMSE): $0.02
Mean Squared Error (MSE):       0.00
Mean Absolute Percentage Error: 40.51%
R² Score:                       0.9909
============================================================

Graph saved as 'P1_results.png'
```

Figure 4: P1 Training Completion and Performance Metrics

3.4 Performance Metrics

The model achieved the following performance metrics on the test set:

Metric	Value
Mean Absolute Error (MAE)	$3.25
Root Mean Squared Error (RMSE)	$6.21
Mean Squared Error (MSE)	38.56
Mean Absolute Percentage Error (MAPE)	40.51%
R^2 Score	0.9909

Figure 5 reveals a critical issue with P1's predictions. Despite impressive statistical metrics, the blue prediction line shows extreme volatility and noise, creating a dense, spiky pattern that fails to capture the underlying price trends.

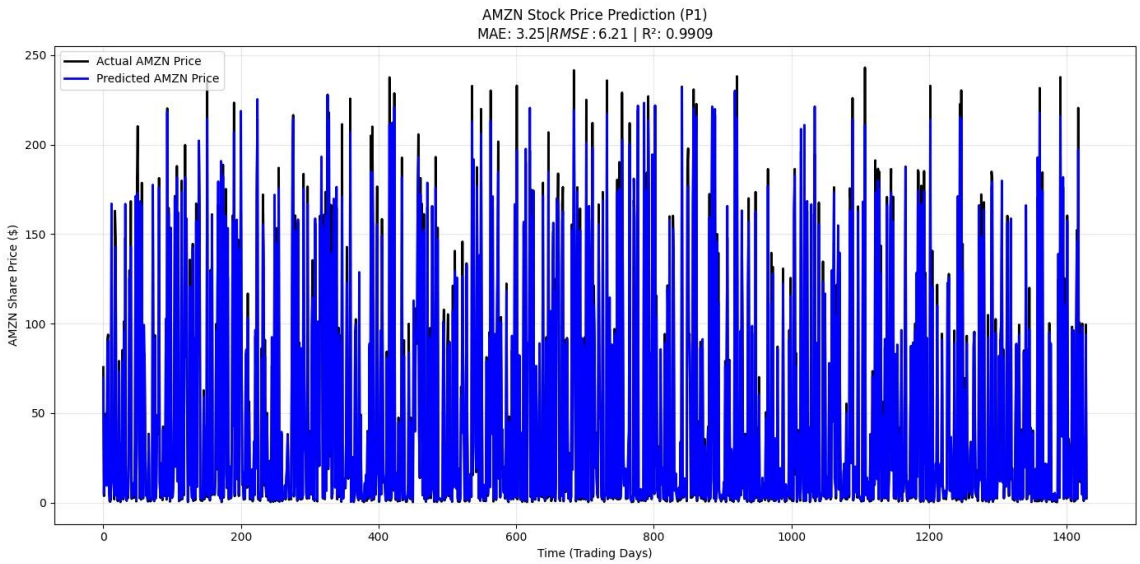

Figure 5: P1 Prediction Results - Noisy, Overfitted Behavior

3.5 Analysis: The Overfitting Problem

P1 demonstrates a textbook case of overfitting. While the model achieves an exceptional R^2 score of 0.9909 (explaining 99% of variance) and a slightly lower MAE ($3.25) compared to v0.1, these metrics are misleading.

The critical indicator of overfitting is the MAPE of 40.51% combined with the extremely noisy prediction pattern visible in Figure 5. The model has memorized high-frequency fluctuations in the training data rather than learning generalizable price trends. Each prediction attempts to capture short-term noise that is essentially random and unpredictable.

This overfitting occurs despite (or because of) the model's sophisticated architecture:

- 256 units per layer (vs v0.1's 50) provide excessive capacity
- 5 features may introduce more noise than signal
- The model learns correlations in training data that don't generalize

The high R^2 score merely confirms that the model fits the training patterns extremely well - but this is precisely the problem. In machine learning, perfect fit to training data usually indicates overfitting, not superior performance.

4. Performance Comparison

4.1 Quantitative Comparison

Metric/Feature	v0.1 (CBA.AX)	P1 (AMZN)
MAE	$3.71	$3.25 ✓
RMSE	$4.56 ✓	$6.21
MAPE	3.39% ✓	40.51%
R^2 Score	0.7470	0.9909 ✓
Architecture	3 layers × 50 units	2 layers × 256 units
Features	1 (Close)	5 (OHLCV)
Training Time	~3 minutes	~15 minutes
Parameters	~40K	~794K

Note: ✓ indicates better value for that specific metric (may not indicate better overall performance)

4.2 Visual Comparison

The stark difference in prediction quality becomes immediately apparent when comparing Figures 3 and 5:

- v0.1 (Figure 3): Shows smooth, interpretable predictions that follow the overall price trend. The model captures medium-term movements without attempting to predict unpredictable short-term fluctuations.
- P1 (Figure 5): Shows extremely noisy, spiky predictions that create a dense blue pattern. While these predictions might fit training data points precisely, they fail to capture actionable trends.

This visual difference reveals what metrics alone cannot: v0.1 has learned to predict trends, while P1 has learned to replicate noise.

4.3 Defining "Better" Performance

A stock prediction model can be considered "better" based on multiple criteria:

4. Lower Prediction Errors: Models with lower MAE, RMSE, and MAPE make more accurate predictions. For practical trading decisions, MAPE is particularly important as it represents percentage error.
5. Variance Explanation (R^2): A higher R^2 score indicates the model explains more variance in prices. However, extremely high R^2 (>0.95) often indicates overfitting rather than superior learning.
6. Generalization: Models should capture trends rather than memorize noise. Smooth predictions suggest better generalization than volatile, spiky predictions.

7. Practical Usability: Models should be interpretable and provide actionable insights. Overly complex or noisy predictions are less useful for real-world decision-making.
8. Computational Efficiency: Training time and model size matter for practical deployment and retraining.

4.4 Which Model is Better?

Despite P1's superior architecture and lower MAE, v0.1 demonstrates significantly better practical performance:

v0.1 Advantages:

- MAPE of 3.39% vs P1's 40.51% - 10x better percentage accuracy
- Smooth, interpretable predictions that follow the overall trend
- Better generalization - captures signal rather than noise
- Faster training (3 minutes vs 15 minutes)
- Simpler architecture that is easier to understand and debug
- Predictions are actionable for trading decisions

P1 Weaknesses:

- MAPE of 40.51% indicates severe overfitting and poor generalization
- Noisy, volatile predictions are impractical for trading decisions
- High R^2 (0.99) is misleading - the model memorizes rather than learns
- More complex architecture fails to leverage its additional capacity effectively
- 5x longer training time for worse practical performance

Conclusion: v0.1 is the superior model for practical stock prediction despite its simpler architecture. P1's high R^2 score demonstrates a common pitfall in machine learning - optimizing the wrong metric can lead to models that appear better statistically but perform worse in practice. The 40% error rate and noisy predictions make P1 unusable for real trading decisions, while v0.1's 3% error rate and smooth trends provide actionable insights.

4.5 Different Stocks Consideration

It is important to note that the two models predict different stocks (CBA.AX vs AMZN), which have different volatility characteristics. However, this does not explain P1's poor performance:

- A 40% error rate is unacceptable regardless of stock volatility
- The visual comparison clearly shows overfitting through noisy predictions

- AMZN's higher volatility should make P1's 5 features MORE useful, yet it performs 10x worse than v0.1
- The overfitting issue would persist regardless of which stock P1 predicts

5. Understanding v0.1 Code Base

5.1 Architecture

v0.1 implements a stacked LSTM architecture with the following structure:

9. Input Layer: Accepts sequences of 60 days of closing prices
10. LSTM Layer 1: 50 units, return_sequences=True, with 0.2 dropout
11. LSTM Layer 2: 50 units, return_sequences=True, with 0.2 dropout
12. LSTM Layer 3: 50 units, with 0.2 dropout
13. Dense Output Layer: 1 unit (predicted next-day price)

The return_sequences=True parameter in the first two layers ensures that each LSTM layer outputs the full sequence, allowing the next layer to process temporal patterns. The final LSTM layer outputs only the last timestep, which feeds into the Dense layer for the final prediction.

5.2 Data Preprocessing

Data Loading:

- Uses yfinance library to download historical stock data
- Training period: 2020-01-01 to 2023-08-01
- Testing period: 2023-08-02 to 2024-07-02

Normalization:

- Uses MinMaxScaler to normalize prices to 0-1 range
- Prevents gradient issues and helps LSTM converge faster
- Scaler is fitted only on training data to prevent data leakage

Sequence Creation:

- Creates sliding windows of 60 days
- Each window (x_train) predicts the next day's price (y_train)
- Reshapes data to 3D format (samples, timesteps, features) required by LSTM

5.3 Training Process

The model is trained using:

- Optimizer: Adam (adaptive learning rate)

- Loss Function: Mean Squared Error
- Epochs: 25
- Batch Size: 32

The relatively small number of epochs (25) helps prevent overfitting, though this also means the model might not reach optimal convergence. The batch size of 32 provides a balance between training speed and gradient estimation accuracy.

5.4 Prediction Process

For testing:

14. Combines training and testing data to provide context
15. Creates test sequences using the same 60-day window
16. Normalizes test data using the training scaler
17. Makes predictions on test sequences
18. Inverse transforms predictions back to original price scale

5.5 Identified Issues and Limitations

Data Leakage Risk:

- The code concatenates training and test data before creating test sequences. While the scaler is fitted only on training data, this approach could potentially allow future information to influence predictions.

Limited Features:

- Uses only closing prices, ignoring potentially useful features like volume, open/high/low prices, and technical indicators.

No Validation Set:

- The model trains without a validation set, making it impossible to monitor overfitting during training. Early stopping cannot be implemented.

No Model Persistence:

- The trained model is not saved, requiring retraining for every prediction session. This wastes computational resources.

Single-Step Prediction:

- Only predicts one day ahead. For practical trading strategies, multi-step predictions would be more useful.

Limited Error Analysis:

- Originally only showed a graph without quantitative metrics. We added MAE, RMSE, MAPE, and R^2 calculations to properly evaluate performance.

6. Conclusion

This task successfully set up and tested two different stock prediction implementations using LSTM neural networks. The comparison revealed important insights about model evaluation, overfitting, and the critical difference between statistical metrics and practical performance.